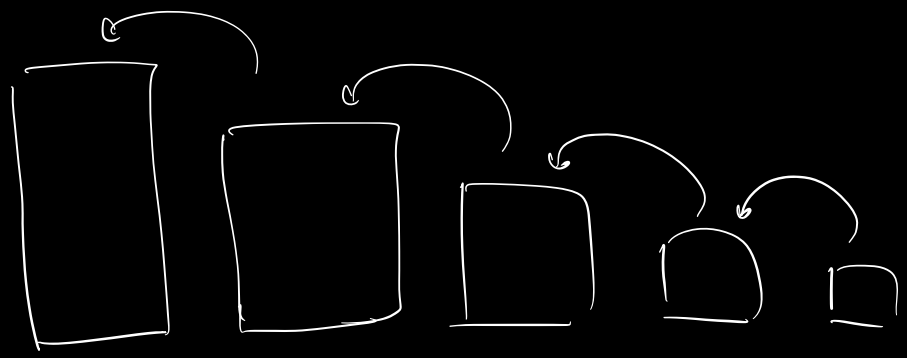
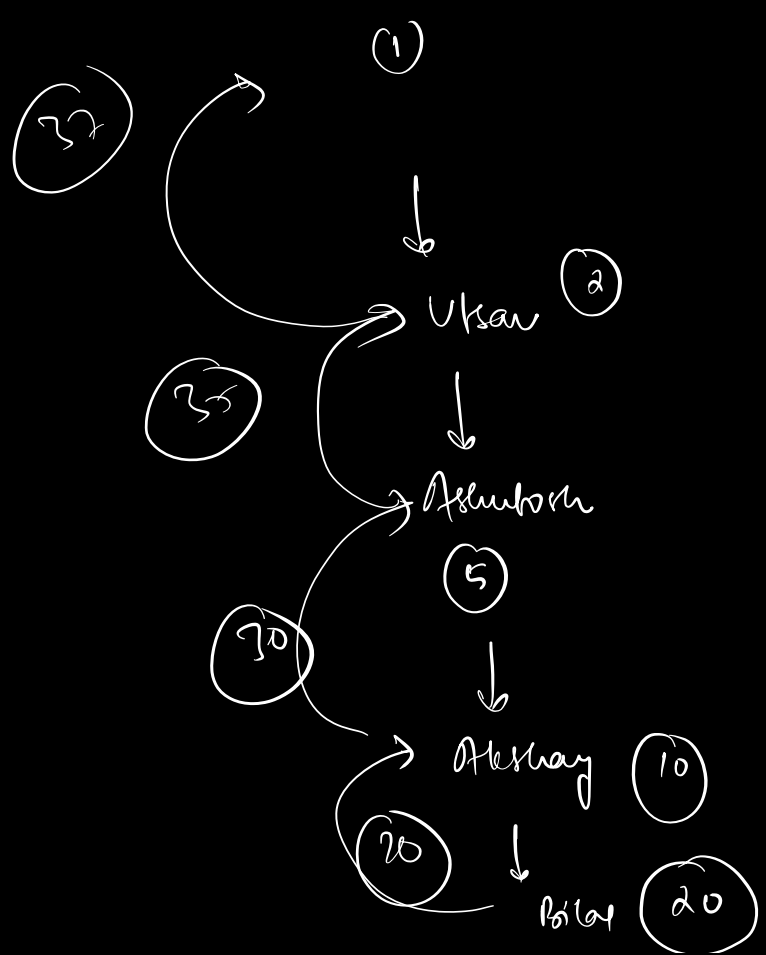
Recursion :- Solving a problem with smaller sub problem



Parivesh



Sum of N natural nos.

sum(50) → 1 + 2 + 3 + 4 + 5 + 6 + ---- + 49 + 50.

↓

sum(49) + 50

sum(1) = 1

sum(N) = N + sum(N-1)

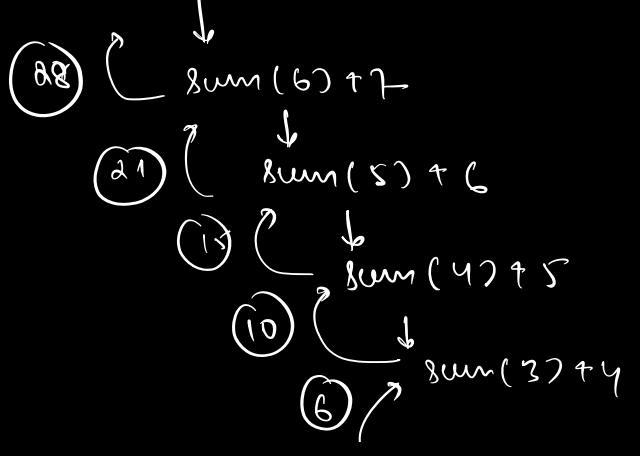
sum(1) = 1.

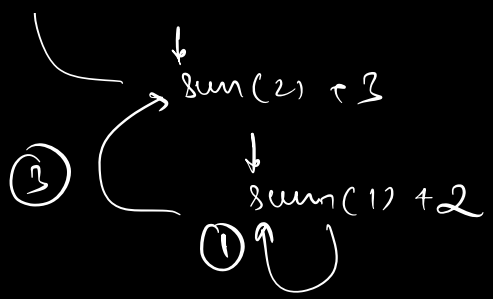
⟶ Sum of N natural recursively

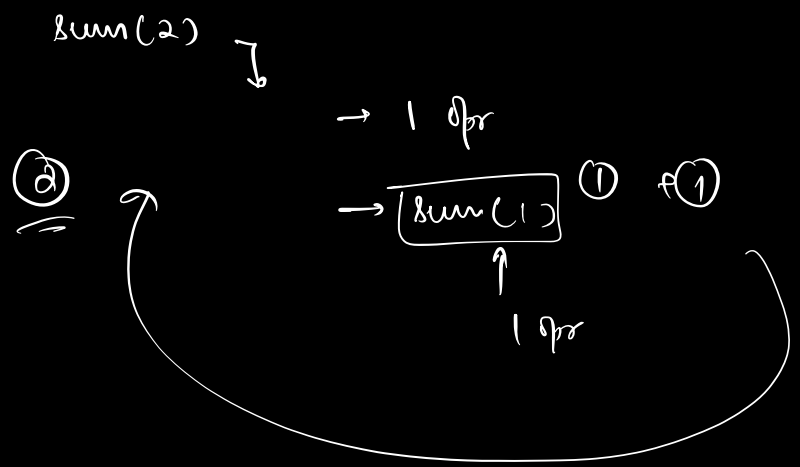
```
int sum(N) {
    if(N==1) ] O(1)
        return 1;
    return sum(N-1)+N; } O(1)
}
```

O(1)

sum(7) ⟶ sum(7) → 28







∴ Time complexity in any recursive code = no. of times func call was made × TC of each recursive call.

TC ⇒ O(N)

SC ⇒ O(N) ← recursion call stack.

example

```
int sum(N) {
    if(N==1)
        return 1;
    for(i=0; i<N; i++) { print(n)
    }
    return sum(N-1)+N;
}
```

no. of times × TC of each call

N × O(N)

⇒ O(N²)

⇒ factorial of N nos. :-

fact(5) ⇒ 5 × 4 × 3 × 2 × 1 (fact(4))

fact(4) ⇒ 4 × 3 × 2 × 1

fact(1) = 1

fact(0) = 1

[T(N) = T(N-1) + 1]

fact(N) = N × fact(N-1)

```
int fact(N) {
    if(N<=1) ] O(1)
        return 1;
    return fact(N-1)*N; } O(1)
}
```

TC ⇒ O(N)
SC ⇒ O(N)

⇒ fibonacci series

0	1	2	3	4	5	6	7	8	9	10
0	1	1	2	3	5	8	13	21	34	55

find Nth fibonacci nos.

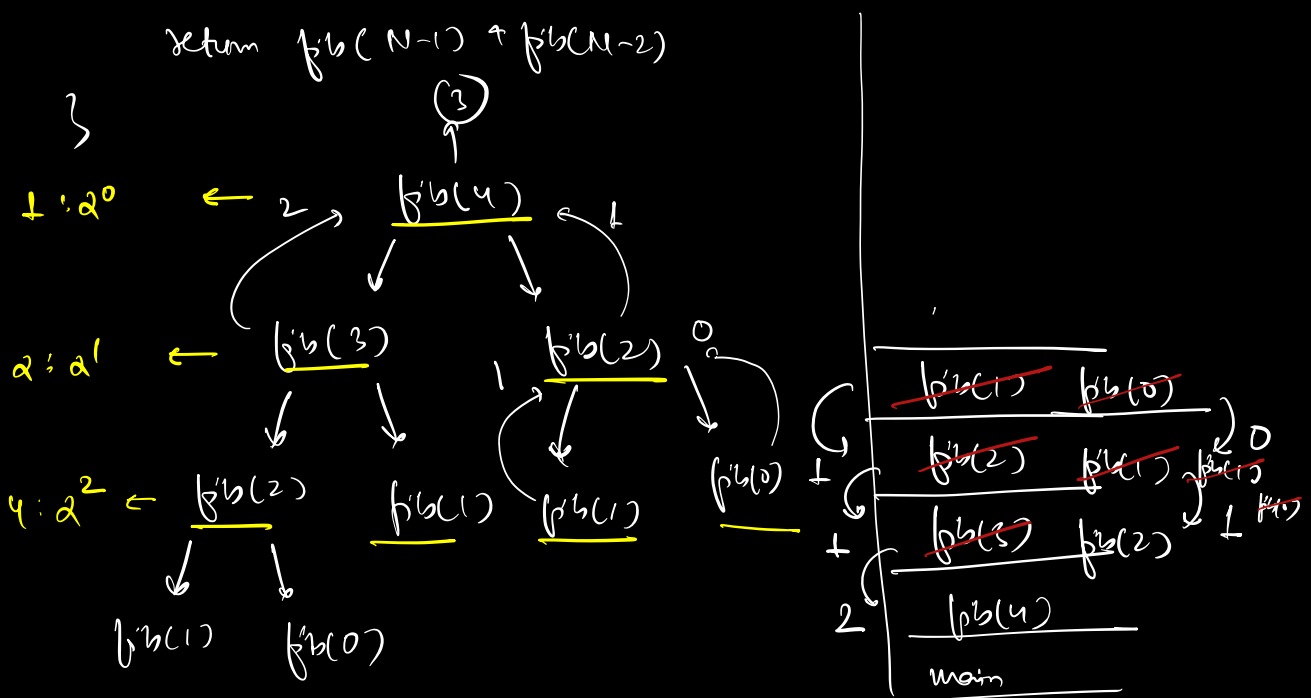
fib(0) = 0

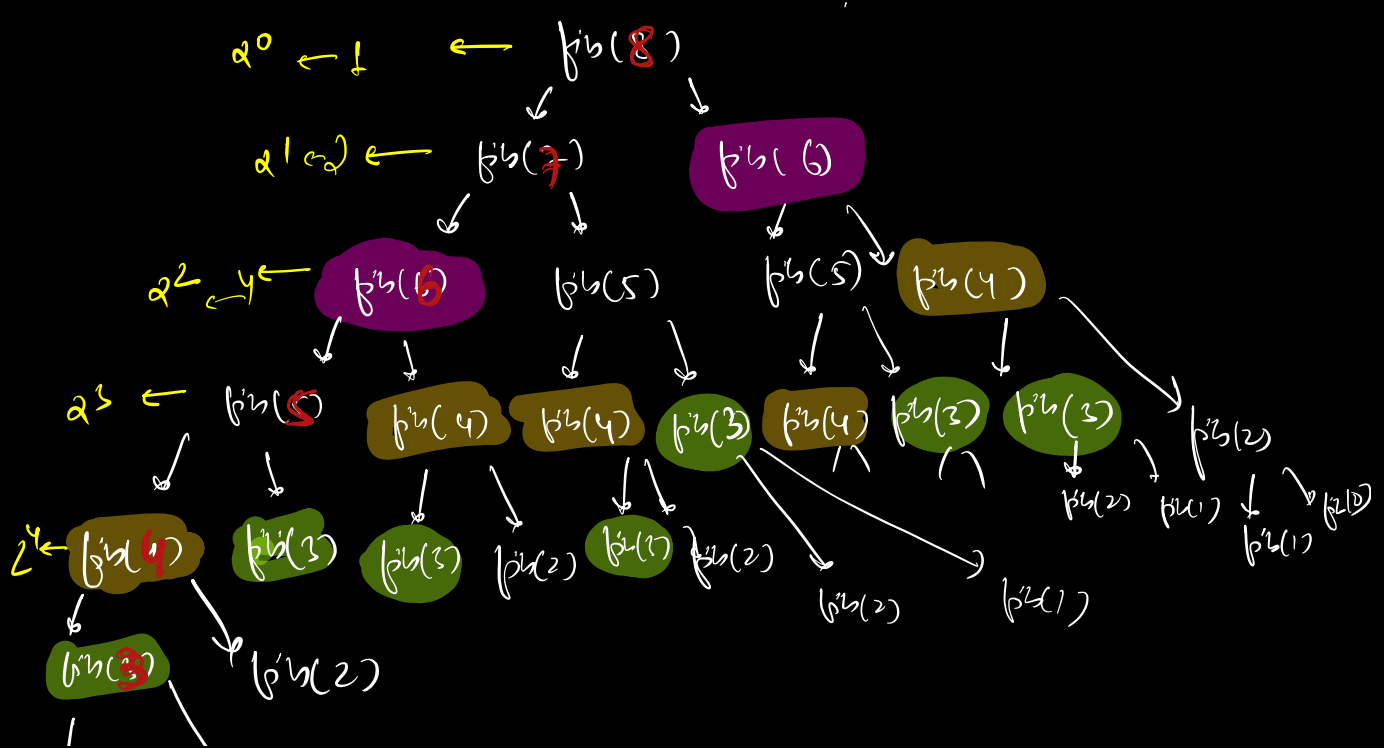
fib(1) = 1

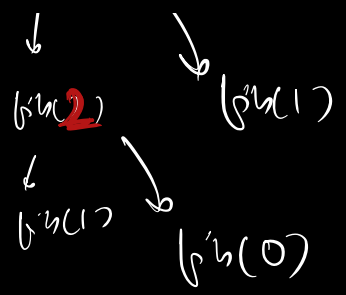
fib(N) = fib(n-1) + fib(n-2)

```
int fib(N) {
    if (N<=1 && N>=0) {
        return N;
    return fib(N-1) + fib(N-2)
}
```

$[T(N) = T(N-1) + T(N-2) + 1]$







GP → 2^0 2^1 2^2 2^3 — —

Sum of GP ⇒ $\frac{a(r^N - 1)}{r - 1}$

a = first term
r = ratio
N = no. of term

⇒ $\frac{2^0(2^N - 1)}{(2-1)}$

⇒ $2^N - 1$

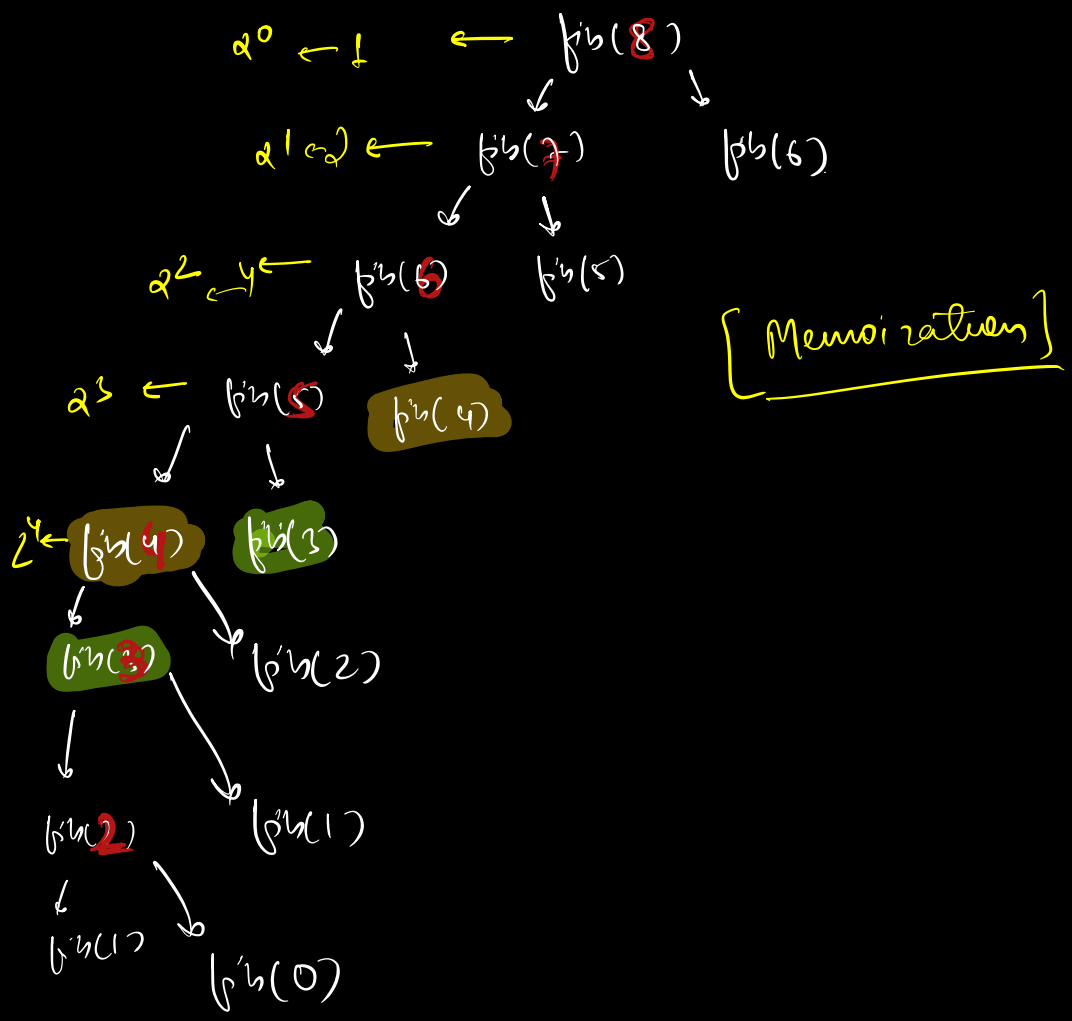
2^0
2^1
2^2
2^3
.
.
.
2^{N-1}

sum of all calls = $2^N - 1$

TC ⇒ $O(2^N)$

SC ⇒ $O(N)$

	0	1	2	3	4	5	6	7
arr[N] ⇒	0	1	1	2	3	5	8	13



⇒ GCD → greatest common divisor

[gcd(a, b) → gcd(a, b%a) a <= b

if (a == 0)

gcd = b

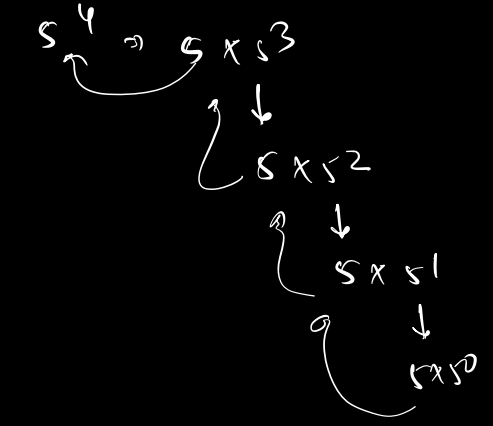
```
int gcd(a,b){
    if(a==0)
        return b
    return gcd(b%a, a);
}
```

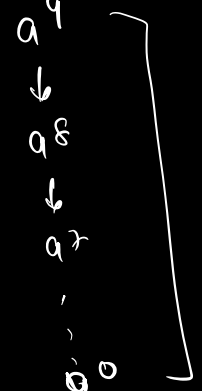
- power

pow(a, N)

```
int pow(a,N){
    if(N==0)
        return 1
    return pow(a,N-1)*a
}
```

TC ⇒ O(N)
SC ⇒ O(N)





$a^{10} \Rightarrow a^5 \times a^5$

$a^5 \Rightarrow a^2 \times a^2 \times a$

$a^2 \Rightarrow a \times a$

$a^{16} \Rightarrow a^8 \times a^8$

$a^8 \Rightarrow a^4 \times a^4$

$a^4 \rightarrow a2 \times a2$

$a2 \rightarrow a^1 \times a^1$

if power is even (N)

$a^N \Rightarrow a^{N/2} \times a^{N/2}$

if, power is odd (N)

$a^N \Rightarrow a^{N/2} \times a^{N/2} \times a$

$T(N) = 2T(N/2) + 1$

a^N

↓

$N == 0$, return 1

$N == 1$, return a

N → even

return $a^{N/2} \times a^{N/2}$

N → odd

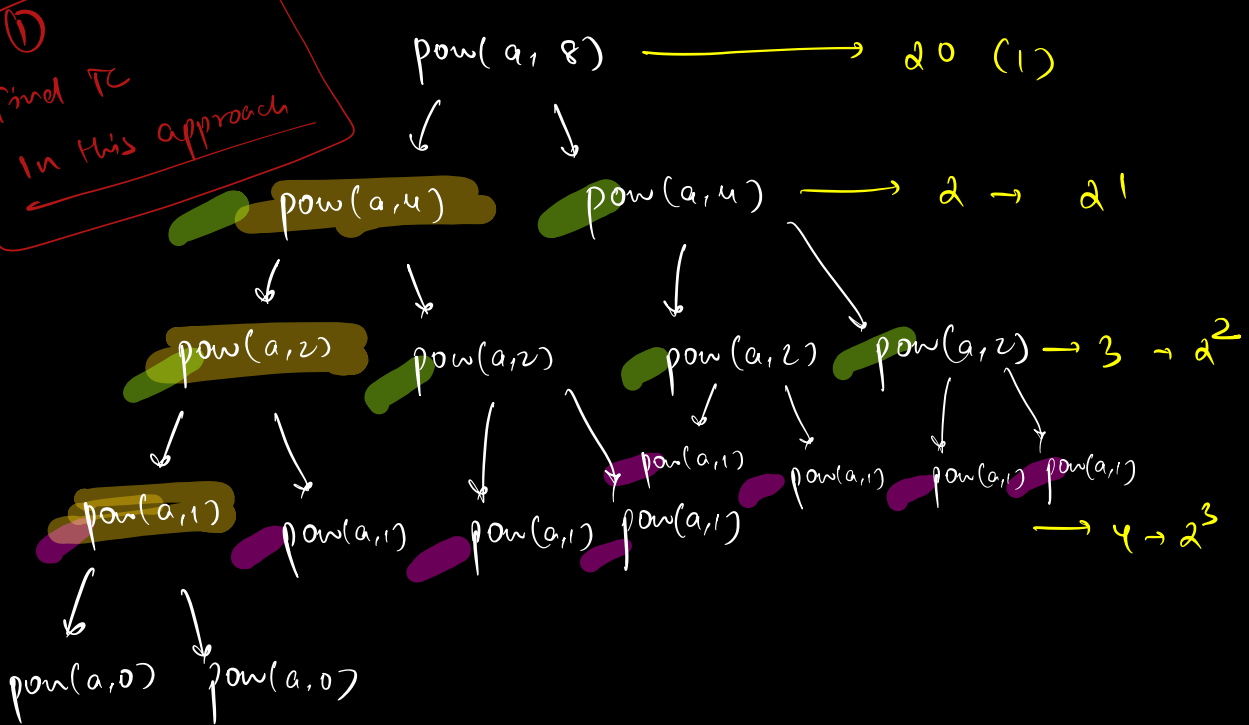
return $a^{N/2} \times a^{N/2} \times a$

$T(N)$

```
int pow(a, N) {
  if (N == 0)
      return 1
  if (N & 1 == 0)        T(N/2)   T(N/2)
      return pow(a, N/2) * pow(a, N/2)
  else
  return pow(a, N/2) * pow(a, N/2) *
}
```

a

① Find TC in this approach



```
int pow(a, N){
    if(N==0)
        return 1
    x = pow(a, N/2);
    if(N&1==0)
        return x*x
    else
        return x*x*a;
}
```

T(N)

memoisation variable

T(N/2)

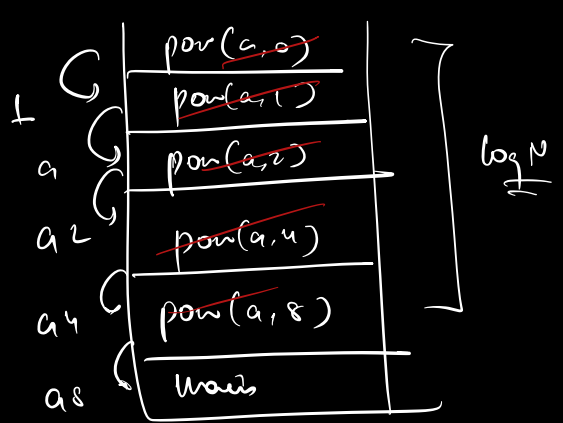
pow(a,8)
↓
pow(a,4)
↓
pow(a,2)
↓
pow(a,1)
↓
pow(a,0)

TC ⇒ O(log N)
SC ⇒ O(log N)

(11) Can SC ever be greater TC.

prove

(a,8)



① $T(N) = T(N/2) + 1$

$= T(N/4) + 1 + 1$

② $= T(N/4) + 2$

$= T(N/8) + 1 + 2$

③ $= T(N/8) + 3$

After k steps

$T(N) = T\left(\frac{N}{2^k}\right) + k$

$T(1) = 1$

$T(N/2) = T(N/4) + 1$

$T(N/4) = T(N/8) + 1$

if $T\left(\frac{N}{2^k}\right) = 1$

then $\frac{N}{2^k} = 1$

$\Rightarrow N = 2^k$

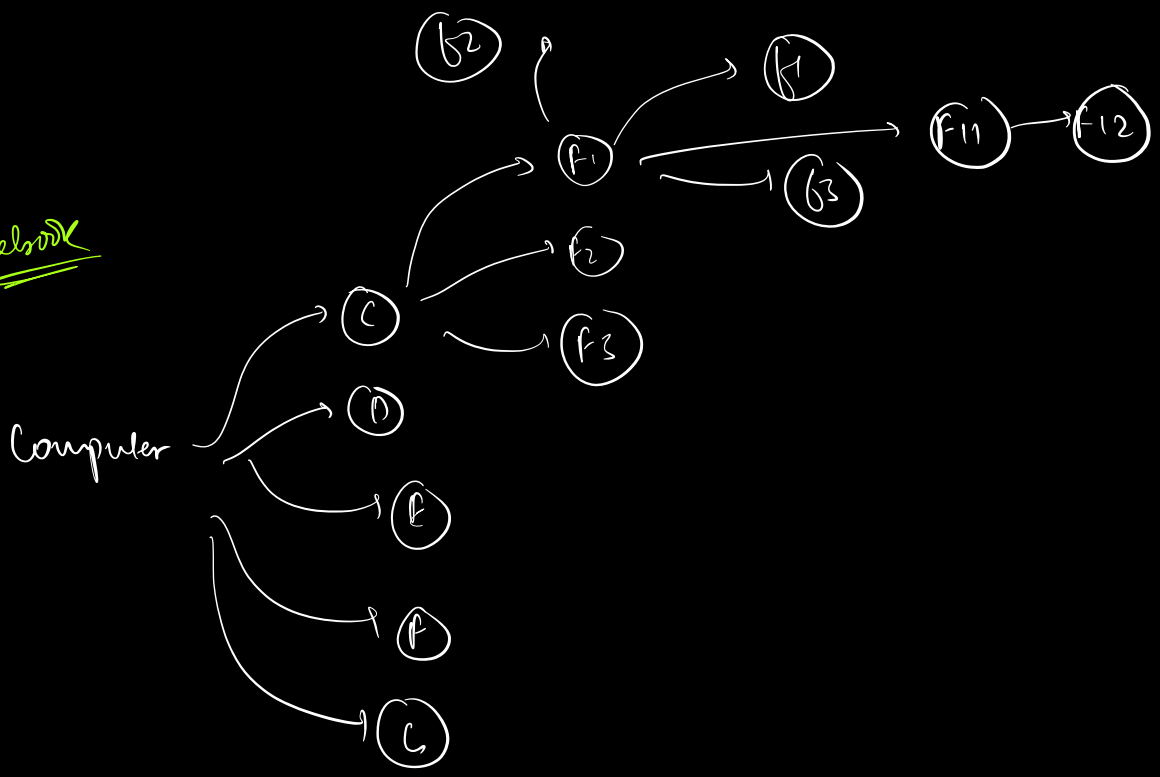
$\Rightarrow \log_2 N = \log_2 2^k$

$\Rightarrow \log_2 N = k \log_2 2$ (1)

$\Rightarrow k = \log_2 N$

TC $\Rightarrow O(\log_2 N)$

SC $\Rightarrow O(\log_2 N)$



find a file

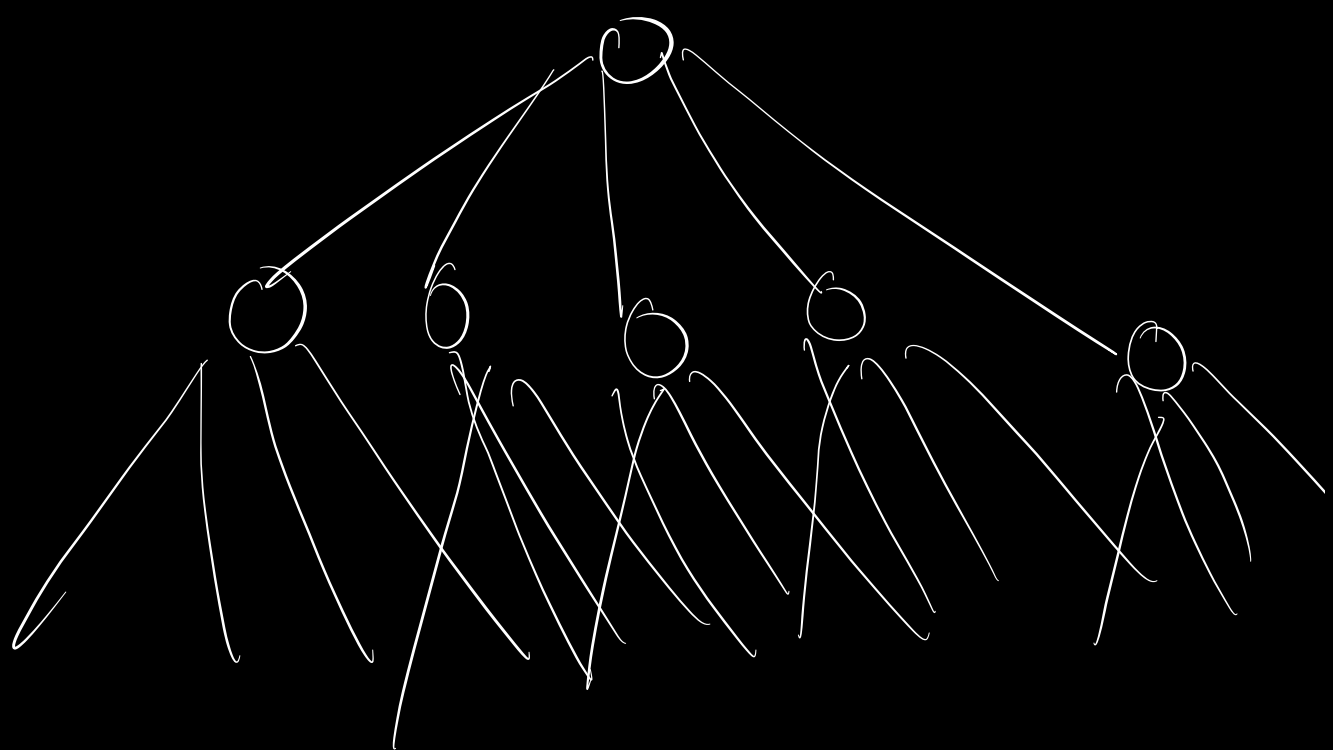
List of files

1) getAllFiles (directoryName)
2) getAllFolders (directoryName)

List of folder

```
boolean search(fileName, root) {
    files = getAllFiles(root);
    for(i=0; i<N; i++) {
        if(files[i] == fileName)
            return true
    }
```

```
folders = getAllFolder(root)
for( folder f : folders ){
    if(search( fileName, f ))
                return true;
}
return false
}
```



Find the TC if, maxm depth from root = k
maxm files in a folder = N
maxm folder in a folder = M.